

Introduzione alla Sicurezza nello Sviluppo Software

- Obiettivi del corso
 - Comprendere l'importanza di integrare la sicurezza fin dalle prime fasi di sviluppo.
 - Ridurre la superficie di attacco e applicare il principio del minimo privilegio.
 - Implementare misure di difesa in profondità.
 - Integrare controlli di sicurezza nel ciclo di vita dello sviluppo (SDLC).
- Contenuti principali
 - Riduzione della superficie di attacco.
 - Principio del minimo privilegio.
 - Difesa in profondità.
 - SDLC e sicurezza.
 - Strumenti e metodologie per lo sviluppo di software sicuro.

Riduzione della Superficie di Attacco

- Definizione
 - Ridurre la superficie di attacco significa limitare le potenziali vie di accesso per un attaccante, disabilitando funzioni e servizi non necessari.
- Strategie
 - Disabilitare servizi, porte e funzioni non essenziali.
 - Rimuovere codice legacy o moduli non utilizzati.
 - Configurare in modo restrittivo i servizi esposti.
- Benefici
 - Minore possibilità di sfruttamento delle vulnerabilità.
 - Riduzione dell'impatto in caso di compromissione.

Principio del Minimo Privilegio

- Definizione
 - Ogni utente o processo deve avere solo i privilegi strettamente necessari per svolgere il proprio compito.
- Esempi pratici
 - Assegnare permessi specifici su file e risorse.
 - Creare account con privilegi limitati per i servizi.
 - Utilizzare ruoli e gruppi per segregare le responsabilità.
- Vantaggi
 - Riduce la possibilità che un attaccante, in caso di compromissione, possa accedere a risorse critiche.
 - Limita l'impatto di errori o vulnerabilità nel codice.

Difesa in Profondità

- Definizione
 - Implementare più livelli di controlli di sicurezza per proteggere il sistema, in modo che il fallimento di una misura non comprometta l'intero sistema.
- Livelli di difesa
 - Perimetro: Firewall, sistemi di rilevamento delle intrusioni (IDS/IPS).
 - Applicazione: Autenticazione, autorizzazione, validazione degli input.
 - Dati: Crittografia, backup, gestione degli accessi.
- Vantaggi
 - Maggiore resilienza contro attacchi multi-fase.
 - Possibilità di intercettare e bloccare l'attacco in vari stadi.

SDLC e Integrazione della Sicurezza

- Fasi del ciclo di vita (SDLC) con focus sulla sicurezza
- Requisiti
 - Raccolta e analisi dei requisiti di sicurezza.
 - Definizione degli standard e delle linee guida (es. OWASP Top 10).
- Progettazione
 - Incorporare controlli di sicurezza nell'architettura.
 - Utilizzo di diagrammi di flusso per identificare potenziali punti di attacco.
- Implementazione
 - Scrittura di codice sicuro.
 - Revisione del codice (code review) focalizzata sulla sicurezza.

SDLC e Integrazione della Sicurezza

- Test
 - Test di sicurezza, vulnerability assessment, penetration testing.
- Distribuzione
 - Configurazioni sicure del sistema.
 - Controllo degli accessi e monitoraggio.
- Manutenzione
 - Aggiornamenti regolari e gestione delle patch.
 - Monitoraggio continuo e analisi degli incidenti.
- Strumenti utili
 - Analisi statica e dinamica del codice.
 - Scanner di vulnerabilità.
 - Sistemi di gestione delle patch.

Metodologie e Strumenti per lo Sviluppo di Software Sicuro

- Metodologie
 - Integrazione della sicurezza in ogni fase del processo di sviluppo.
 - Revisione periodica del codice e audit di sicurezza.
 - Formazione continua degli sviluppatori sui temi della sicurezza.
- Strumenti
 - Static Analysis: SonarQube, Bandit (per Python), FindBugs (per Java).
 - Dynamic Analysis: OWASP ZAP, Burp Suite.
 - Gestione dei Bug: Sistemi di ticketing per tracciare e risolvere le vulnerabilità.
- Linee guida e standard
 - Aderenza a standard di codifica sicura.
 - Implementazione di controlli e procedure per la gestione delle patch.

Casi di Studio e Best Practices

- Casi reali
 - Analisi di attacchi noti e loro impatto.
 - Discussione su come la mancanza di misure di sicurezza abbia facilitato l'attacco.
- Best Practices
 - Monitoraggio costante e analisi delle vulnerabilità.
 - Pianificazione di emergenza e risposta agli incidenti.
 - Cultura della sicurezza all'interno dell'organizzazione.

Esempio

- `docker build . -t macchina-sicura`
- `docker run -it --rm -p 2222:22 -p 2121:21 -p 8080:80 -p 2323:23 macchina-sicura`

Checklist

- Analisi iniziale
 - Elencare i servizi attualmente in esecuzione nel container.
 - Verificare quali porte sono esposte e su quali servizi.
- Valutazione dei Servizi
 - ???
- Implementazione delle Modifiche
 - ???
- Documentare le modifiche apportate e l'impatto osservato sulla sicurezza del container.
- Effettuare scansioni di porte per confermare che solo i servizi essenziali sono attivi.
- Dimostrare, tramite report, come la riduzione della superficie d'attacco riduca i rischi di vulnerabilità.

Checklist

- **Analisi iniziale**
 - Elencare i servizi attualmente in esecuzione nel container (SSH, FTP, HTTP, Telnet).
 - Verificare quali porte sono esposte e su quali servizi.
- **Valutazione dei Servizi**
 - HTTP (Apache): Necessario se l'applicazione è web-based.
 - SSH: Necessario solo se è prevista la gestione interattiva remota; altrimenti, va disabilitato.
 - FTP: Necessario solo se l'applicazione richiede il trasferimento file in questo modo; in alternativa, usare metodi più sicuri.
 - Telnet: Inaccettabile in qualsiasi scenario a causa della mancanza di sicurezza.
- **Implementazione delle Modifiche:**
 - Disabilitare o rimuovere i servizi non necessari.
- Documentare le modifiche apportate e l'impatto osservato sulla sicurezza del container.
- Effettuare scansioni di porte per confermare che solo i servizi essenziali sono attivi.
- Dimostrare, tramite report, come la riduzione della superficie d'attacco riduca i rischi di vulnerabilità.